Nayan Modak

cs24btech11043@iith.ac.in | nayan.modak.2006@gmail.com | +91 7428272772 | LinkedIn | GitHub

EDUCATION

Institution	Degree / Board	Passing Year	Score / CGPA
Indian Institute of Technology Hyderabad	B.Tech	2028	8.24
Narayana Junior College	Class XII (Telengana Board of Intermediate Education)	2024	97.00%
Don Bosco School	Class X (Central Board of Secondary Education)	2022	96.40%

SKILLS

- **Programming Languages:** C, C++, Python, Java, OCaml, Assembly(RISC-V), Bash
- **Software & Tools:** Git, Linux (Ubuntu), eBPF / sched_ext (SCX), LaTeX, Docker, Solid Edge

PROJECTS

SCX_CARRY: IPC-Aware eBPF CPU Scheduler | C, eBPF, sched_ext, Linux Kernel | 2026 | github

(Part of Operating Systems II coursework)

- Developed a hot-swappable Linux CPU scheduler using C/eBPF and sched_ext, with kernel scheduling logic and a userspace CLI for real-time monitoring.
- Implemented hardware-counter-based task classification, per-task state in BPF maps, and tiered dispatch queues for Intel hybrid P/E cores.
- Added vruntime prioritization, CPU-hog and micro-burst penalties, cache-affinity fast paths, and starvation-safe fallback paths for predictable latency.
- Built repeatable stress-ng and Counter-Strike 2 benchmarks; improved compute throughput 10.7%, matrix throughput 3.3%, reduced package power 11%, and sustained ~5.3 ms frametimes under 24 background workers.

Highly Concurrent Swap Subsystem for xv6 | C, RISC-V, Systems Programming | 2026 | github

(Part of Operating System II coursework)

- Extended xv6-riscv with eager/lazy heap allocation and disk-backed swapping, handling page faults while preserving PTE permissions across eviction and restore.
- Implemented clock page replacement and a concurrency-safe swap-slot allocator using spinlocks and a strict lock hierarchy.
- Replaced the default scheduler with a 4-level MLFQ using exponential time quanta, interactive-process heuristics, and periodic priority boosts.
- Created SMP stress and integrity tests with 32 processes on 8 cores; verified 1,700+ evictions with zero data corruption, kernel panics, or lock hangs.

Arbitrary-Precision Arithmetic Library | Java, OOP, JUnit, CLI | 2025 | github

(Part of Software Development Fundamentals coursework)

- Designed a modular Java library with AInteger and AFloat classes using string-based representations to support arithmetic beyond native numeric limits.
- Implemented signed addition, subtraction, long multiplication, long division, decimal alignment, precision handling, rounding, and notation conversion.
- Added custom exceptions for division by zero and invalid input, built a command-line interface, and containerized the application with Docker for reproducible setup and execution.

Exception-Handling Extension for MiniML Compiler | OCaml, Compilers, Grammars | 2025 | github

(Part of Software Development Fundamentals coursework)

- Extended the compiler pipeline across lexer, parser, AST, type checking, and a stack-based abstract machine to support integer division and structured exceptions.
- Implemented try-with/raise semantics, runtime fault classification, call-frame unwinding, evaluation-stack cleanup, and nearest-handler matching.
- Implemented runtime error propagation in the stack-based abstract machine, safely unwinding call frames and evaluation stacks to the nearest matching exception handler.

Cross-Platform CPU Performance Optimization | C++, RISC-V, GEM5 | 2025 | github

(Part of Computer Architecture coursework)

- Implemented matrix multiplication and determinant in C++ and manually benchmarked them on gem5/RISC-V, ARM Cortex-A76, and Intel x86-64.
- Optimized kernels using loop interchange, cache tiling, loop unrolling, and compiler auto-vectorization; profiled IPC, cache misses, execution time, and store-buffer stalls.
- Achieved a 2.2x speedup in gem5 and documented architecture-specific correctness/performance trade-offs, including low-ILP limitations of the Bareiss algorithm.

RELEVANT COURSES

- **CS**-Intro to Programming, Software Development Fundamentals, Computer Architecture, Operating System, DBMS, Data Structures, Algorithms, Theory of computation, Compilers-I
- **Maths** - Calculus-I, Calculus-II, Discrete Maths, Probability and Random Variables, Transform Techniques, Linear Algebra, Intro to statistics